Resultados: flujos con y sin buffer en Java

Práctica de medición con System.nanoTime() · n = 1,000,000 · JDK 26 en VS Code

Tabla de resultados (n = 1,000,000)

Con n = 1,000,000 hay tres ejecuciones sin depurador. Las ejecuciones 2 y 3 se corrigieron porque reporte() dividía con un divisor distinto de 1_000_000.0 (÷10 y ×10); la corrida con depurador se descartó.

Flujo	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	428.868	389.284	410.660	409.604
BufferedWriter	139.577	120.304	120.140	126.674
FileReader	551.915	560.555	543.240	551.903
BufferedReader	201.608	202.074	189.360	197.681
FileOutputStream	4494.194	4631.007	4385.250	4503.484
BufferedOutputStream	23.081	19.820	20.790	21.230
FileInputStream	3242.665	3138.331	3096.370	3159.122
BufferedInputStream	17.423	21.761	16.250	18.478

Tabla de resultados (n = 100,000)

Medido en un entorno de prueba (Linux, Java 21, un núcleo), no en tu equipo: sirve para ver la tendencia, no para comparar valores absolutos con la tabla de 1,000,000. Tres ejecuciones sin depurador; el texto movió 1,688,890 caracteres y el binario 100,000 bytes.

Flujo	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	233.561	160.373	156.133	183.356
BufferedWriter	60.403	67.948	85.212	71.188
FileReader	213.112	340.407	356.858	303.459
BufferedReader	86.119	87.904	84.525	86.183
FileOutputStream	69.774	77.207	76.542	74.508
BufferedOutputStream	20.359	15.833	15.720	17.304
FileInputStream	40.961	37.361	37.554	38.625

Flujo	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
BufferedInputStream	12.841	19.608	13.696	15.382

Tabla de resultados (n = 10,000,000)

Medido en el mismo entorno de prueba (Linux, Java 21, un núcleo), no en tu equipo. Tres ejecuciones sin depurador; el texto movió 188,888,890 caracteres y el binario 10,000,000 bytes.

Flujo	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	1362.597	1081.356	1202.818	1215.590
BufferedWriter	514.497	538.458	582.007	544.987
FileReader	7423.777	7328.090	7486.615	7412.827
BufferedReader	3570.098	3564.013	3500.434	3544.848
FileOutputStream	3186.703	3486.901	3354.663	3342.756
BufferedOutputStream	209.796	221.303	205.874	212.324
FileInputStream	2722.407	2735.875	2756.635	2738.306
BufferedInputStream	210.108	208.127	207.650	208.628

Qué muestran los datos

Pregunta 1: buffer vs. directo

El buffer gana en los ocho casos, pero en magnitudes muy distintas:

Par	Sin buffer (ms)	Con buffer (ms)	Aceleración
FileWriter → BufferedWriter	409.6	126.7	≈ 3.2x
FileReader → BufferedReader	551.9	197.7	≈ 2.8x
FileOutputStream → BufferedOutputStream	4503.5	21.2	≈ 212x
FileInputStream → BufferedInputStream	3159.1	18.5	≈ 171x

En binario, un millón de llamadas al sistema (una por byte) se reducen a unas 122 de 8 KB, y de ahí salen las mejoras de cientos de veces. En texto la mejora es menor porque FileWriter y FileReader ya llevan un buffer interno de 8 KB en su codificador; el BufferedWriter solo ahorra llamadas a ese codificador.

Pregunta 3: texto vs. Binario

El resultado es contraintuitivo. movió 17,888,890 caracteres y binario tan sólo 1,000,000 bytes, pero FileOutputStream (≈ 4.5 s) era unas 11 veces más lento que FileWriter (≈ 0.41 s).que domina el costo no el volumen de datos, sino el número de llamadas al sistema operativo: FileOutputStream realiza 1 millón de operaciones, mientras que FileWriter solo hace unas miles gracias a su buffer interno.

Con buffer a ambos lados el orden se invierte: el binario con buffer es el más rápido (≈ 21 ms) y el texto con buffer queda en ≈ 127 ms, porque además de mover más datos tiene que codificar caracteres y construir String en cada iteración.

Pregunta 2: escala y lectura vs. escritura

El tiempo crece de forma casi lineal solo a partir de 1,000,000; con 100,000 domina el calentamiento de la JVM. La tabla usa promedios de tres ejecuciones en el entorno de prueba (Linux, Java 21, un núcleo), los tres valores de n en el mismo equipo.

Flujo	n = 100,000 (ms)	n = 1,000,000 (ms)	n = 10,000,000 (ms)
FileWriter	183.4	442.0	1,215.6
BufferedWriter	71.2	117.4	545.0
FileReader	303.5	819.7	7,412.8
BufferedReader	86.2	355.7	3,544.8
FileOutputStream	74.5	348.1	3,342.8
BufferedOutputStream	17.3	40.9	212.3
FileInputStream	38.6	275.9	2,738.3
BufferedInputStream	15.4	37.8	208.6

Entre 100 000 y 1 000 000 los tiempos crecen entre 1,7x y 7,1x, aunque n se multiplica por 10. Entre 1.000.000 y 10.000.000, los flujos de lectura de texto y los de bytes sin buffer crecen de forma lineal entre 9,0x y 10,0x. BufferedWriter, BufferedOutputStream y BufferedInputStream crecen entre 4.6x y 5.5x, y FileWriter solo 2.8x; una explicación probable es que el JIT y los costos fijos de arranque importan más cuando el tiempo total es corto.

La lectura y la escritura no se comportan igual en el texto. Con n = 10,000,000, FileReader es unas 6 veces más lento que FileWriter (7,413 ms vs 1,216 ms) y BufferedReader unas 6,5 veces más lento que BufferedWriter, pues la escritura hace una llamada por cadena (10,000,000 llamadas) y la lectura carácter a carácter hace una por carácter (188,888,890 llamadas). En binario sí se comportan de forma simétrica: cada operación mueve un byte, y FileInputStream y FileOutputStream están en el mismo orden de magnitud (≈ 2,7 s y 3,3 s), al igual que BufferedInputStream y BufferedOutputStream (≈ 209 ms y 212 ms).

Calidad de la medición

- Las tres ejecuciones son consistentes entre sí (diferencias de 3 a 34 %, mayores en los tiempos más cortos), lo que da confianza en las conclusiones cualitativas.
- Cada método se ejecuta una sola vez por corrida y sin calentamiento, así que el primer método (FileWriter) sale algo penalizado por el JIT. Conviene mencionarlo como limitación, junto con la recomendación de usar JMH para mediciones rigurosas.

Preguntas 4 y 5

Pregunta 4: tamaño del buffer por defecto

Las cuatro clases usan un buffer de 8192 elementos, y todas permiten cambiarlo con un segundo argumento en el constructor.

Clase	Tamaño por defecto
`BufferedOutputStream`	8192 bytes
`BufferedInputStream`	8192 bytes
`BufferedWriter`	8192 caracteres
`BufferedReader`	8192 caracteres

```
new BufferedOutputStream(new FileOutputStream(ruta), 65536);
```

```
new BufferedReader(new FileReader(ruta), 32768);
```

Un buffer mayor reduce las llamadas al sistema, pero con rendimientos decrecientes, y uno demasiado grande desperdicia memoria.

Pregunta 5: readLine() en BufferedReader y no en BufferedInputStream

readLine() es del mundo de los caracteres: una línea termina en \n, \r o \r\n, y reconocerlo necesita datos ya decodificados como texto. BufferedInputStream no da ningún significado a los bytes que maneja, por lo que un 0x0A puede ser simplemente un dato más, como parte de un entero o una imagen.

BufferedReader permite leer texto línea a línea con while ((linea = br.readLine()) != null), con la codificación del charset resuelta. Los bytes habría que acumularlos y decodificarlos a mano, con el riesgo de partir un carácter multibyte por la mitad, una ñ en UTF-8, por ejemplo.

Conclusión personal

Los buffers reducen el número de accesos costosos al sistema operativo agrupando muchas operaciones pequeñas en pocas grandes. En mis mediciones con n = 1,000,000, el binario con buffer fue entre 170 y 210 veces más rápido que sin buffer, mientras que en texto la mejora fue de unas 3 veces, porque FileWriter y FileReader ya incluyen un buffer interno. Aun así, BufferedReader aporta comodidad con readLine().